

OpenF1 React + Spring Boot Architecture Guide (Full Version)

Overview

Recommended structure:

```
backend/  
├─ controller/  
├─ service/  
├─ client/  
├─ dto/  
└─ config/
```

```
frontend/  
├─ pages/  
├─ components/  
├─ api/  
└─ hooks/
```

Backend - controller/

Responsibility:

- Expose REST endpoints
- Receive HTTP requests
- Validate input parameters
- Call services
- Return responses

Should NOT:

- Call OpenF1 directly
- Contain business logic
- Perform calculations

Example:

```
@RestController  
@RequestMapping("/api/drivers")
```

```

public class DriverController {

    private final DriverService driverService;

    public DriverController(DriverService driverService) {
        this.driverService = driverService;
    }

    @GetMapping
    public List<DriverDto> getDrivers(
        @RequestParam Integer sessionKey) {

        return driverService.getDrivers(sessionKey);
    }
}

```

Flow:

Frontend



Controller



Service

Backend - service/

Responsibility:

- Business logic
- Determine which OpenF1 endpoints to call
- Combine data from multiple sources
- Perform calculations
- Handle caching decisions

Simple example:

```

public List<DriverDto> getDrivers(Integer sessionKey)

```

Complex example:

```
public DriverComparisonDto compareDrivers(  
    int sessionKey,  
    int driver1,  
    int driver2)
```

Possible flow:

1. Retrieve lap data
2. Retrieve stint data
3. Calculate averages
4. Determine fastest lap
5. Build comparison response

Backend - client/

Responsibility:

- Communicate with OpenF1
- Manage URLs
- Manage query parameters
- Handle external API integration

Example:

@Component

```
public class OpenF1Client {  
  
    private final RestClient restClient;  
  
    public List<OpenF1DriverResponse> getDrivers(  
        Integer sessionKey) {  
  
        return restClient.get()  
            .uri(uriBuilder ->  
                uriBuilder  
                    .path("/v1/drivers")  
                    .queryParams("session_key", sessionKey)  
                    .build())  
            .retrieve()  
            .body(new ParameterizedTypeReference<>() {});  
    }  
}
```

Service usage:

```
openF1Client.getDrivers(sessionKey);
```

Backend - dto/

Responsibility:

- Define data transferred between layers

Incoming OpenF1 DTO:

```
public class OpenF1DriverResponse {  
  
    private Integer driverNumber;  
    private String fullName;  
    private String teamName;  
  
}
```

Outgoing API DTO:

```
public class DriverDto {  
  
    private Integer number;  
    private String name;  
    private String team;  
  
}
```

Mapping flow:

OpenF1 Response DTO

↓

Service

↓

Frontend DTO

This prevents your frontend from being tightly coupled to OpenF1.

If OpenF1 changes tomorrow:

```
{  
  "driver_name": "Max Verstappen"  
}
```

to

```
{  
  "full_name": "Max Verstappen"  
}
```

only your client mapping changes.

Backend - config/

Responsibility:

- RestClient configuration
- CORS configuration
- Cache configuration
- Security configuration

RestClient Example:

```
@Configuration  
public class OpenF1Config {  
  
    @Bean  
    RestClient openF1RestClient() {  
        return RestClient.builder()  
            .baseUrl("https://api.openf1.org")  
            .build();  
    }  
}
```

Cache Example:

```
@Configuration  
@EnableCaching  
public class CacheConfig {  
}
```

CORS

```
@Configuration
public class CorsConfig {
}
```

Cache configuration

```
@Configuration
@EnableCaching
public class CacheConfig {
}
```

Security configuration

```
@Configuration
public class SecurityConfig {
}
```

Frontend - pages/

Responsibility:

- Route-level screens
- Correspond directly to URLs
- Orchestrate components

Examples:

```
pages/
├─ HomePage.jsx
├─ DriversPage.jsx
├─ DriverDetailsPage.jsx
├─ SessionsPage.jsx
└─ RacePage.jsx
```

React Router:

```
<Route path="/drivers" element={<DriversPage />} />
```

Frontend - components/

Responsibility:

- Reusable UI components
- Focus on presentation

Examples:

components/

└─ DriverCard.jsx

└─ DriverTable.jsx

└─ LapTimeChart.jsx

└─ SessionSelector.jsx

└─ Navbar.jsx

Example:

```
function DriversPage() {  
  
  const { data } = useDrivers();  
  
  return (  
    <>  
      <Navbar />  
      <DriverTable drivers={data} />  
    </>  
  );  
}
```

Frontend - api/

Responsibility:

- Communicate with Spring Boot backend
- Frontend should never call OpenF1 directly

Architecture:

React

↓

Spring Boot

↓

OpenF1

Example:

```
import axios from "axios";

export async function getDrivers(sessionKey) {

  const response = await axios.get(
    `/api/drivers`,
    {
      params: {
        sessionKey
      }
    }
  );

  return response.data;
}
```

Suggested structure:

```
api/
├─ driverApi.js
├─ sessionApi.js
├─ raceApi.js
└─ telemetryApi.js
```

Frontend - hooks/

Responsibility:

- Reusable frontend logic
- Data fetching
- State management
- UI abstraction

Example:

```
export function useDrivers(sessionKey) {

  const [drivers, setDrivers] = useState([]);
```



```
useEffect(() => {  
  
    getDrivers(sessionKey)  
        .then(setDrivers);  
  
    }, [sessionKey]);  
  
    return drivers;  
}
```

Usage:

```
const drivers = useDrivers(sessionKey);
```

Complete Request Flow

```
DriversPage  
↓  
useDrivers()  
↓  
driverApi.getDrivers()  
↓  
GET /api/drivers  
↓  
DriverController  
↓  
DriverService  
↓  
OpenF1Client  
↓  
OpenF1 API  
↓  
OpenF1 Response DTO  
↓  
DriverService Mapping  
↓  
DriverDto  
↓  
Controller Response  
↓
```

React Hook



Component Rendering

Recommended Expanded Structure

backend/

- └─ controller/
- └─ service/
- └─ client/
- └─ dto/
 - └─ request/
 - └─ response/
 - └─ openf1/
- └─ mapper/
- └─ exception/
- └─ config/
- └─ cache/

frontend/

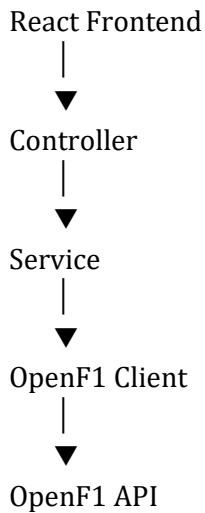
- └─ pages/
- └─ components/
 - └─ drivers/
 - └─ races/
 - └─ sessions/
 - └─ common/
- └─ api/
- └─ hooks/
- └─ types/
- └─ utils/
- └─ context/

Why:

- mapper = DTO conversion
- exception = global exception handling
- cache = OpenF1 caching
- types = TypeScript interfaces/types
- utils = shared helpers
- context = global state management

EXAMPLE:

The typical flow looks like this:



1. Frontend calls the Controller

For example, your React app might do:

```
const response = await fetch(  
  "http://localhost:8080/api/drivers"  
);
```

```
const drivers = await response.json();
```

This sends an HTTP request to your Spring Boot backend.

2. Controller receives the request

```
@RestController  
@RequestMapping("/api/drivers")  
public class DriverController {  
  
    private final DriverService driverService;  
  
    public DriverController(DriverService driverService) {  
        this.driverService = driverService;  
    }  
  
    @GetMapping  
    public List<DriverDto> getDrivers() {
```

```
        return driverService.getDrivers();
    }
}
```

When Spring sees:

GET /api/drivers

it executes:

```
driverService.getDrivers();
```

The controller generally shouldn't contain much logic. Its job is to:

- Receive requests
 - Validate inputs
 - Call a service
 - Return a response
-

3. Service contains business logic

```
@Service
public class DriverService {

    private final OpenF1Client openF1Client;

    public DriverService(OpenF1Client openF1Client) {
        this.openF1Client = openF1Client;
    }

    public List<DriverDto> getDrivers() {
        return openF1Client.fetchDrivers();
    }
}
```

The service decides:

- Which OpenF1 endpoint to call
- Whether to combine multiple API calls
- Whether to cache data
- Whether to transform the response

For example:

```
public DriverProfileDto getDriverProfile(int driverNumber) {

    DriverDto driver =
        openF1Client.fetchDriver(driverNumber);

    List<PitStopDto> pitStops =
        openF1Client.fetchPitStops(driverNumber);

    return DriverProfileMapper.combine(
        driver,
        pitStops
    );
}
```

The controller doesn't need to know any of this.

4. Client talks to OpenF1

```
@Component
public class OpenF1Client {

    private final RestClient restClient;

    public OpenF1Client(RestClient restClient) {
        this.restClient = restClient;
    }

    public List<DriverDto> fetchDrivers() {

        return restClient.get()
            .uri("/drivers")
            .retrieve()
            .body(new ParameterizedTypeReference<>() {});
    }
}
```

This is the only class that knows:

- OpenF1's URL
- OpenF1 query parameters

- OpenF1 response formats

If OpenF1 changes their API tomorrow, you mostly update the client layer.

How do they "communicate"?

Through **Spring Dependency Injection**.

When the application starts, Spring creates the objects for you.

```
@RestController
public class DriverController {

    private final DriverService driverService;

    public DriverController(DriverService driverService) {
        this.driverService = driverService;
    }
}
```

Spring sees:

```
@Service
public class DriverService
```

and automatically injects it into the controller.

Similarly:

```
@Service
public class DriverService {

    private final OpenF1Client openF1Client;

    public DriverService(OpenF1Client openF1Client) {
        this.openF1Client = openF1Client;
    }
}
```

Spring sees:

```
@Component
public class OpenF1Client
```

and injects that too.

You never write:

```
new DriverService();  
new OpenF1Client();
```

Spring handles object creation and wiring.

Full Request Example

Suppose React loads a Drivers page.

React

```
fetch("/api/drivers")
```

Controller

```
driverService.getDrivers();
```

Service

```
openF1Client.fetchDrivers();
```

Client

```
GET https://api.openf1.org/v1/drivers
```

OpenF1 Response

```
[  
  {  
    "driver_number": 1,  
    "full_name": "Max Verstappen"  
  }  
]
```

Client returns

```
List<DriverDto>
```

Service returns

```
List<DriverDto>
```

Controller returns

```
List<DriverDto>
```

React receives

```
[
  {
    "driverNumber": 1,
    "fullName": "Max Verstappen"
  }
]
```

For an OpenF1 application, I'd actually recommend having **one OpenF1Client class (or a small set of clients)** and multiple domain services:

DriverController -> DriverService -> OpenF1Client
RaceController -> RaceService -> OpenF1Client
SessionController -> SessionService -> OpenF1Client
WeatherController -> WeatherService -> OpenF1Client

That keeps all external API communication in one place while each service focuses on a specific Formula 1 domain.